

TOWERS OF HANOI

Aim:

To implement the Towers of Hanoi problem using recursion in Prolog.

Objective:

To demonstrate recursive problem solving by moving N disks from one peg to another while following the rules of Towers of Hanoi.

Algorithm:

- ☐ **Start.**
- ☐ **If there is one disk, move it directly.**
- ☐ **For N disks:**
 - **Move N-1 disks from source to auxiliary.**
 - **Move the largest disk from source to destination.**
 - **Move N-1 disks from auxiliary to destination.**
- ☐ **Display each movement.**
- ☐ **Stop.**

Flowchart:

START

|

v

Read N

|

v

Is N = 1?

/ \

Yes No

| |

Move Disk Move N-1

| Source->Aux

| |

| Move Largest

| |

| Move N-1

| Aux->Dest

\ /

v v

Display Moves

|

v

STOP

Prolog Program:

% Planets Database

planet(mercury, terrestrial, 1).

planet(venus, terrestrial, 2).

planet(earth, terrestrial, 3).

planet(mars, terrestrial, 4).

planet(jupiter, gas_giant, 5).

planet(saturn, gas_giant, 6).

planet(uranus, ice_giant, 7).

planet(neptune, ice_giant, 8).

terrestrial(P) :-

planet(P, terrestrial, _).

gas_giant(P) :-

planet(P, gas_giant, _).

ice_giant(P) :-

planet(P, ice_giant, _).

Output:

```
75 ?- consult(['c:/Users/Niharika Preejesh/OneDrive/Documents/tower of hanoi.pl']).
true.

76 ?- hanoi(3, a, c, b).
Move disk from a to c
Move disk from a to b
Move disk from c to b
Move disk from a to c
Move disk from b to a
Move disk from b to c
Move disk from a to c
true ▲
```

Result:

Thus, the Towers of Hanoi problem is successfully implemented using recursion in Prolog.